

VR寿司弾丸

注文に応じて、握って、投げる。VR寿司サービングゲーム

作品概要	VR空間で、来店した客の注文に応じて寿司を作り、投げて提供するゲーム
開発期間	2025年11月～2026年1月（約3ヶ月）
開発体制	3人でのチーム開発
使用技術	Unity / C# / SteamVR Plugin（Interaction System） / Git

01 概要

VR空間で、来店した客の注文に応じて寿司を作り、投げて提供するゲームです。シャリとネタを両手でつかんで組み合わせ、完成した寿司を客に届けます。

注文どおりに提供するとスコアを獲得できます。また、プレイを妨害する特殊な客（クレーマー）も登場し、滞在している間はスコアが減点され続けるため、どの客を優先して対応するかを判断する、状況判断が求められるゲーム性を取り入れました。

02 担当部分

3人チームで開発し、私は主にゲームプレイ全般のプログラムを担当しました。担当した実装範囲は以下のとおりです。

- VR環境における寿司の生成、つかみ・投げ操作の実装
- 注文UI・タイマーUI（頭上表示・残り時間ゲージ）の実装
- 客のスポン、空席判定、座席移動、着席処理の実装
- 客の移動・着席アニメーションの制御
- スコアシステムおよび注文の正誤判定の実装
- リザルト画面におけるランク表示・プレイ振り返り演出の実装

03 工夫した点

- 説明なしでも遊べる直感的な操作**：シャリとネタを両手でつかんで組み合わせる、現実の動作に近いインタラクションにすることで、チュートリアルなしでも操作が伝わるようにしました。
- 瞬時に状況を把握できる頭上UI**：注文内容はアイコンで表示し、残り時間は緑→黄→赤に変化するゲージで表現。プレイ中に視線を止めずに判断できるようにしました。
- 自然なゲーム進行**：客の来店→席への移動→着席→注文開始までの状態遷移を実装し、複数の客が同時に登場しても流れが破綻しない構成にしました。
- プレイ後の達成感を高める演出**：スコア集計・ランク表示に加え、プレイ中の画面を自動でキャプチャし、リザルト画面で振り返れる簡易リプレイ演出を実装しました。

04 課題と解決

CASE 01 肥大化した注文システムのリファクタリング

変更前	注文の生成・タイマー・UI更新・正誤判定・演出・スコア反映が、1つのクラスに集中していました。
問題	クラスが肥大化し、メニュー追加や仕様変更のたびに複数箇所の修正が必要でした。また、寿司の種類を文字列で比較していたため、タイプミスによる不具合が発生するリスクもありました。
変更後	責務ごとにクラスを分離しました。 CustomerOrderWithTimer → 注文の進行・正誤判定 CustomerOrderView → 注文UIの表示 SushiOrderData → 種類・アイコン・注文音声のデータ管理 あわせて寿司の種類の比較を文字列からenum（SushiKind）中心に変更し、比較の安全性を高めました。既存のVR投擲コンポーネントは文字列のインターフェースを使用しているため、境界部分で文字列をenumへ変換する構成にし、既存コードとの互換性を維持しています。さらに、寿司の受け渡し処理の入口を一本化し、正誤判定を注文管理側に集約することで、1個の寿司が二重に判定される問題を防ぎました。

CASE 02 複数の客の来店から注文開始までを安定してつなげる

課題	客のスポン後空席に判定し、移動・着席・注文開始までを自然につなげる必要がありました。
解決	座席ごとに使用状態を持たせ、スポン時に空席を取得して割り当てる仕組みを実装しました。スポン位置はNavMesh上に補正し、到着後は座席の位置・向きに合わせて着席アニメーションを再生します。座席への到着時に注文と制限時間を開始することで、同じ座席への重複割り当てや注文開始タイミングのずれを防ぎました。また、席の予約・解放処理を一本化し、移動中に客が破棄された場合でも席が使用中のまま残らないようにしました。

CASE 03 シーンをまたぐスコア管理とメモリ管理の改善

課題	シーンをまたいでスコアを保持するため、ScoreManagerをSingletonとDontDestroyOnLoadで実装したところ、シーン再ロード時にUI参照が切れる問題が発生しました。また、プレイ振り返り用のTexture2Dは、リストのClear()だけではネイティブメモリが解放されないまま残る問題がありました。
解決	シーン再ロード時に、新しいシーン側のUI参照を既存インスタンスへつなぎ直す処理を実装し、参照切れを解消しました。Texture2Dは使用が終わった時点で各フレームを明示的にDestroy()してから破棄するように改善しました。保存枚数の上限超過時に古いフレームから破棄する処理と合わせ、メモリ使用量が増え続けにくい構成にしています。

05 クラス構成と主要スクリプト

注文システム	
SushiOrderData 注文データ → CustomerOrderWithTimer 注文進行・判定 → CustomerOrderView UI表示	表示する値を渡す
客の流れ	
CustomerSpawner 客のスポン → CustomerSitting 席へ移動・着席 → CustomerOrderWithTimer 注文開始	
SCRIPT	役割
CustomerSpawner.cs	客のスポン、空席判定、NavMesh上への位置補正、座席への誘導
CustomerSitting.cs	客の移動・着席・注文開始までの状態管理、座席の予約・解放
CustomerOrderWithTimer.cs	注文の生成・制限時間の管理・正誤判定・スコアへの反映
CustomerOrderView.cs	頭上の注文UI（アイコン・残り時間ゲージ）の表示専門クラス
SushiOrderData.cs	寿司1種類分の注文データ（種類・アイコン・注文音声）の定義
SushiThrowable.cs	寿司のつかみ・投擲（手の速度推定）・客への受け渡し処理
ScoreManager.cs	スコア集計・ハイスコア保存・プレイ画面の自動キャプチャを担うシングルトン

本作品では、VRならではの直感的な操作と、客の行動・注文・スコア判定を連動させたテンポのよいゲーム進行を目指しました。開発中にコードの肥大化という問題に気づき、自分のコード構成を見直して責務ごとに分離・整理することで、「動くコード」を「保守しやすいコード」へ改善する経験を積むことができました。

- AI活用について
- 本説明書のレイアウト・配色の検討にAIを活用しました。
 - 自分で実装したソースコードのコメントについて、日本語への翻訳や表現の調整に一部AIを活用しました。
 - ゲーム内の店舗看板の一部に、AIで生成した画像を使用しています。